

Traversal on Trees

DAY 16F
24/12/2025

In PreOrder traversal, Each node is processed before either of its Subtrees. This is the Simplest traversal to understand.

However, Even though Each node is processed before the Subtrees, it still requires that Some Information must be maintained While moving down the tree.

In the Example Below, 1 is pre processed first, then the left Subtree, and this is followed By Right Subtree.

Therefore, processing must return to the right Subtree after finishing the processing of the left Subtree.

To move to the right Subtree after processing the left Subtree, We must maintain the root information.

The obvious ADT for Such information is a stack. Because of its LIFO structure. It is possible to get the information about the right Subtrees back in the Reverse order.

PreOrder traversal is Defined as follows

- Visit the root
- Traverse the left Subtree In PreOrder
- Traverse the right Subtree In PreOrder

```
Run | Debug | Run main | Debug main
public static void main(String[] args) {

    TreeNode root = new TreeNode(data: 1);
    root.left = new TreeNode(data: 2);
    root.right = new TreeNode(data: 3);

    root.left.left = new TreeNode(data: 4);
    root.left.right = new TreeNode(data: 5);

    root.right.left = new TreeNode(data: 6);
    root.right.right = new TreeNode(data: 7);

}
```

Recursive approach

```
static void preOrder(TreeNode root){

    if(root == null) return;

    System.out.print("<-> " + root.data );
    preOrder(root.left);
    preOrder(root.right);

}
```

Time & Space Complexity $O(n)$

In ORDER TRAVERSAL

In In ORDER Traversal the root is Visited between the Subtrees.

- Traverse the left subtree InOrder
- Visit the root
- Traverse the Right subtree InOrder

```
static void inOrder(TreeNode root){  
    if(root == null) return;  
  
    inOrder(root.left);  
    System.out.print( "->" + root.data );  
    inOrder(root.right);  
}
```

Time Complexity $O(n)$

Space Complexity $O(n)$

4 2 5 1 6 3 7 InOrder

Post ORDER Traversal

In Post ORDER Traversal, the root is Visited after Both Subtrees, post order traversal is defined as follows:

- Traverse the left subtree
- Traverse the Right subtree
- Visit the root.

```
static void postOrder(TreeNode root){  
    if(root == null) return;  
  
    postOrder(root.left);  
    postOrder(root.right);  
    System.out.print( "->" + root.data );  
}
```

Time Complexity $O(n)$

Space Complexity $O(n)$

